FAST GRAPH-BASED ALGORITHMS FOR IMAGE SEGMENTATION

Jenny Petrova

Mentor Yue Shen

Department of Mathematics

Florida State University

OVERVIEW

- Introduction to Graph Theory
- 2-Way Graph Partitioning
- Algebraic Multigrid Method (with Heavy Edge Coarsening)
- Applications: Image Segmentation

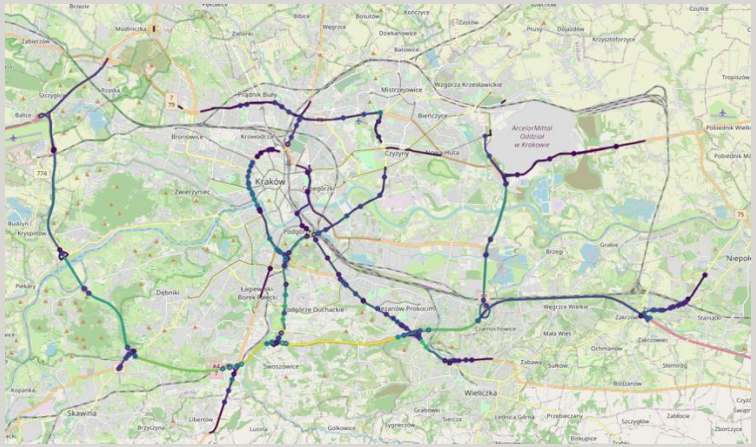
INTRODUCTION TO GRAPH THEORY

INTRODUCTION TO GRAPH THEORY – SOCIAL NETWORK ANALYSIS

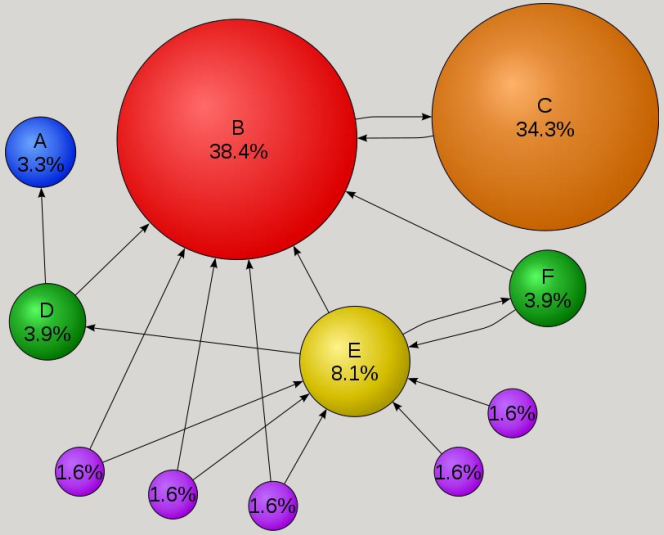
INTRODUCTION TO GRAPH THEORY – MORE EXAMPLES

Neural Network

Transportation Network



Google's PageRank Algorithm



Protein-Protein Interactions (PPIs)

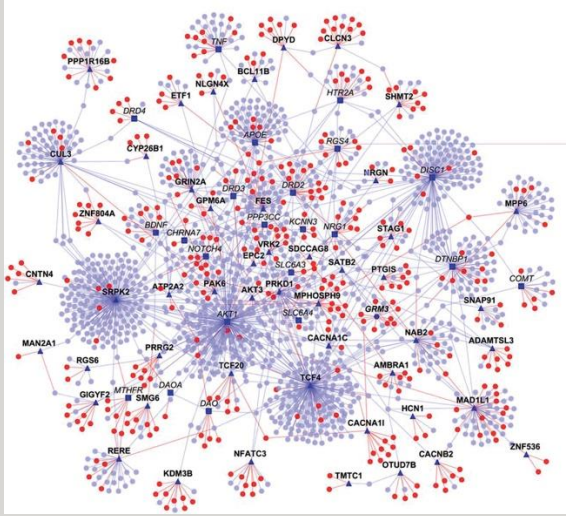
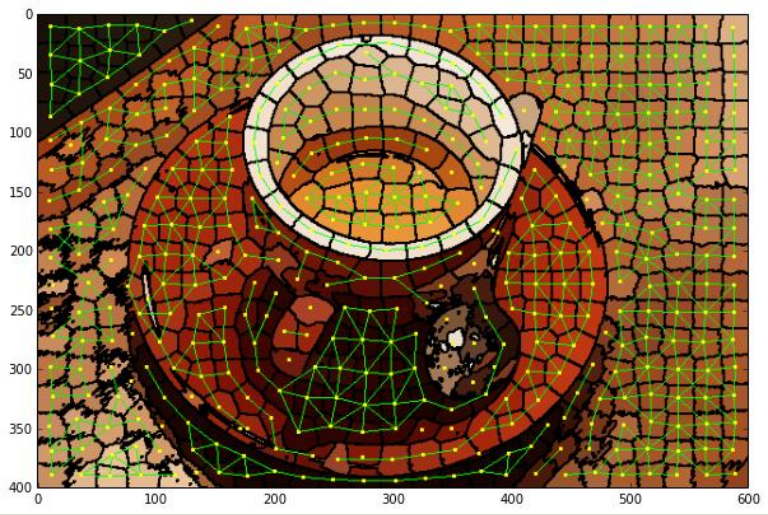


Image Segmentation



Graph theory is the study of graphs – mathematical structures used to model pairwise relations between objects.

Spectral graph theory is the study of the properties of a graph in relation to the characteristic polynomial, eigenvalues, and eigenvectors of matrices associated with the graph, particularly the Laplacian matrix.

NOTATION

Suppose $G = (V, E, W)$ consists of three finite sets:

Vertex set V,

$$V \neq \emptyset$$
$$v_i \in V$$

Edge set E,

$$e_{ij} = (v_i, v_j) \in E$$

and Weight set W

$$w_{ij} \in W$$

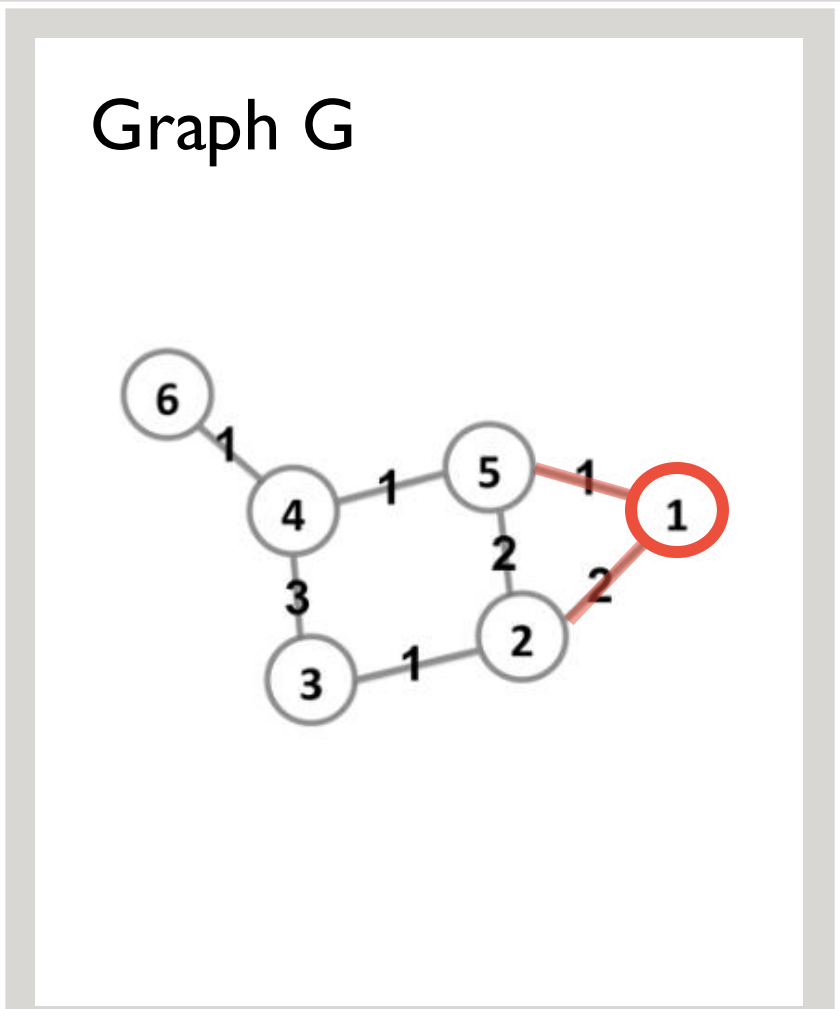
Degree $d_i = d(v_i)$: the number of edges connected to v_i

Weighted Degree $\delta_i = \delta(i) = \sum w_{ij}$

<u>Example</u>: v_1

Degree $d_1 = d(v_1) = 2$

Weighted Degree $\delta_1 = \delta(1) = 1 + 2 = 3$



ADJACENCY MATRIX

Suppose $G = (V, E, W)$ is a connected undirected graph.

Unweighted Adjacency Matrix *A*:

$$A_{ij} = \begin{cases} 1, & \text{if } e_{ij} \in E \\ 0, & \text{otherwise} \end{cases}$$

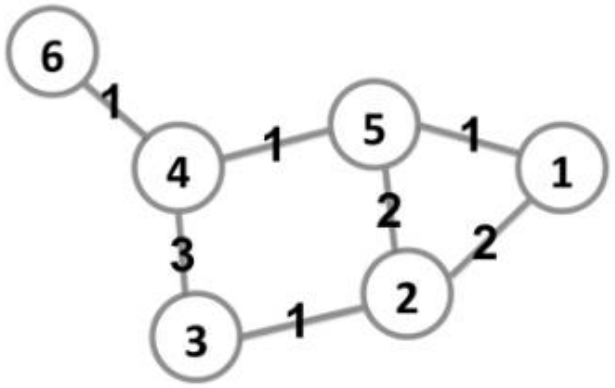
Weighted Adjacency Matrix *W*:

$$W_{ij} = \begin{cases} w_{ij}, & \text{if } e_{ij} \in E \\ 0, & \text{otherwise} \end{cases}$$

Adjacency Matrix for Graph G:

$$W = \begin{bmatrix} 0 & 2 & 0 & 0 & 1 & 0 \\ 2 & 0 & 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 3 & 0 & 0 \\ 0 & 0 & 3 & 0 & 1 & 1 \\ 1 & 2 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Graph G



DEGREE MATRIX

Suppose $G = (V, E, W)$ is a connected undirected graph.

Recall $\delta_i = \sum w_{ij}$

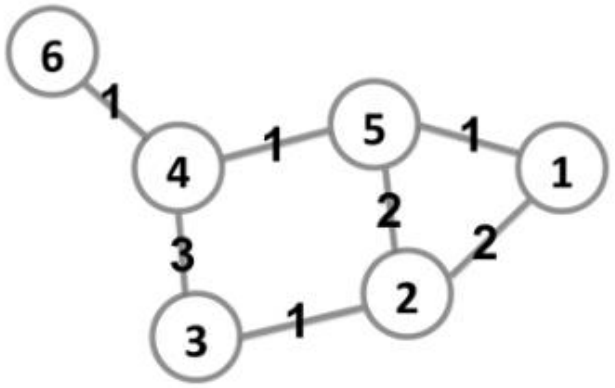
Degree Matrix *D*:

$$D_{ij} = \begin{cases} \delta_i, & i = j \\ 0, & i \neq j \end{cases}$$

Degree Matrix for Graph G:

$$D = \begin{bmatrix} 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 0 \\ 0 & 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Graph G



LAPLACIAN MATRIX

Suppose $G = (V, E, W)$ is a connected undirected graph.

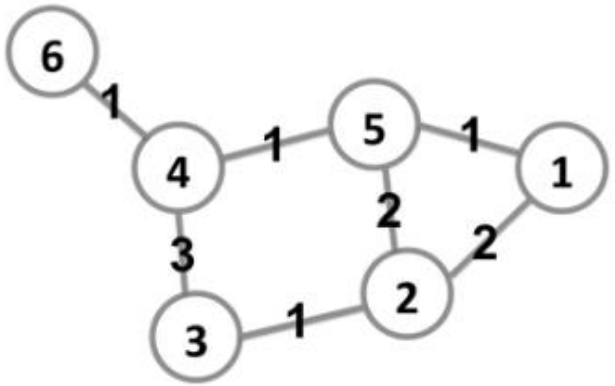
Laplacian Matrix *L* : $L = D - W$

$$W = \begin{bmatrix} 0 & 2 & 0 & 0 & 1 & 0 \\ 2 & 0 & 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 3 & 0 & 0 \\ 0 & 0 & 3 & 0 & 1 & 1 \\ 1 & 2 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 0 \\ 0 & 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$L = D - W = \begin{bmatrix} 3 & -2 & 0 & 0 & -1 & 0 \\ -2 & 5 & -1 & 0 & -2 & 0 \\ 0 & -1 & 4 & -3 & 0 & 0 \\ 0 & 0 & -3 & 5 & -1 & -1 \\ -1 & -2 & 0 & -1 & 4 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{bmatrix}$$

Graph G



LAPLACIAN MATRIX

Suppose $G = (V, E, W)$ is a connected undirected graph.

Laplacian Matrix *L* : $L = D - W$

$$W = \begin{bmatrix} 0 & 2 & 0 & 0 & 1 & 0 \\ 2 & 0 & 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 3 & 0 & 0 \\ 0 & 0 & 3 & 0 & 1 & 1 \\ 1 & 2 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 0 \\ 0 & 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$L = \begin{bmatrix} 3 & -2 & 0 & 0 & -1 & 0 \\ -2 & 5 & -1 & 0 & -2 & 0 \\ 0 & -1 & 4 & -3 & 0 & 0 \\ 0 & 0 & -3 & 5 & -1 & -1 \\ -1 & -2 & 0 & -1 & 4 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{bmatrix}$$

Note:

- L is symmetric
- $L\vec{1} = \vec{0}$

Proof:

$L\vec{1}$

$$= \begin{bmatrix} 3 & -2 & 0 & 0 & -1 & 0 \\ -2 & 5 & -1 & 0 & -2 & 0 \\ 0 & -1 & 4 & -3 & 0 & 0 \\ 0 & 0 & -3 & 5 & -1 & -1 \\ -1 & -2 & 0 & -1 & 4 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 3 - 2 - 1 \\ -2 + 5 - 1 - 2 \\ -1 + 4 - 3 \\ -3 + 5 - 1 - 1 \\ -1 - 2 - 1 + 4 \\ -1 + 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

We can rewrite as $L\vec{1} = \vec{0} = 0 \cdot \vec{1}$

$$L\vec{1} = \begin{bmatrix} 3 & -2 & 0 & 0 & -1 & 0 \\ -2 & 5 & -1 & 0 & -2 & 0 \\ 0 & -1 & 4 & -3 & 0 & 0 \\ 0 & 0 & -3 & 5 & -1 & -1 \\ -1 & -2 & 0 & -1 & 4 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = 0 \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

Definition. Let A be an $n \times n$ matrix.

If $Av = \lambda v$ for a nonzero vector v, we say that λ is the **eigenvalue** for vector v, and that v is an **eigenvector** for λ.

$\therefore$ 0 is an eigenvalue of L, and $\vec{1}$ is the corresponding eigenvector

PROPERTIES OF THE LAPLACIAN MATRIX

Suppose $L \in \mathbb{R}^{n \times n}$ is the Laplacian matrix for a weighted undirected connected graph with eigenvalues $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$ and corresponding eigenvectors $\vec{x}_1, \vec{x}_2, \ldots, \vec{x}_n$

L is symmetric positive semi-definite (all the eigenvalues are non-negative): $0 = \lambda_1 < \lambda_2 \leq \cdots \leq \lambda_n$

λ_2 is the smallest nonzero eigenvalue, called the **Fiedler value** (corresponding eigenvector $\vec{x}_2$ is the **Fiedler vector**)

2-WAY GRAPH PARTITIONING

2-WAY GRAPH PARTITIONING: PROBLEM SETTING

A weighted undirected graph $G = (V, E, W)$ can be partitioned into two disjoint sets, V_1 and V_2 where $V_1 \cup V_2 = V$ and $V_1 \cap V_2 = \emptyset$, by removing edges connecting the two parts:

$$\mathbf{cut}(V_1, V_2) = \sum_{i \in V_1, j \in V_2} w_{ij}$$

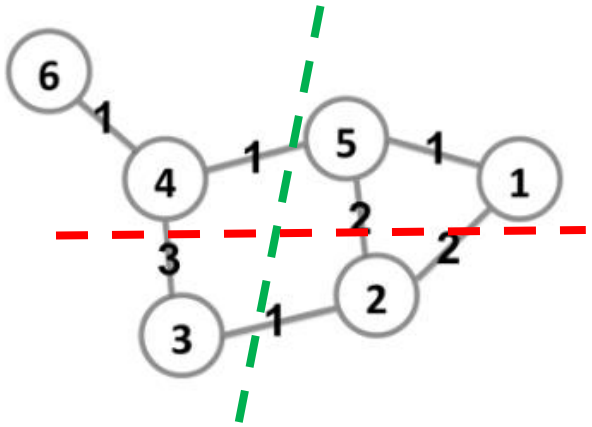
<u>Minimization Problem for $V = V_1 \cup V_2$</u>:

Assume $n = |V|$ is an even number.

min **cut**(V_1, V_2), subject to $|V_1| = |V_2|$

Graph G

$\mathbf{cut}(V_1, V_2) = 2$
$V_1 = \{1,2,5\}$
$V_2 = \{3,4,6\}$



$\mathbf{cut}(V_1, V_2) = 7$
$V_1 = \{1,5,4,6\}$
$V_2 = \{2,3\}$

MINIMIZING THE PARTITION

$$\mathbf{cut}(V_1, V_2) = \sum_{i \in V_1, j \in V_2} w_{ij}$$

<u>Minimization Problem for $V = V_1 \cup V_2$:</u>

Assume $n = |V|$ is an even number.

min $\mathbf{cut}(V_1, V_2)$, subject to $|V_1| = |V_2|$

<u>Equivalent Minimization Problem for $V = V_1 \cup V_2$:</u>

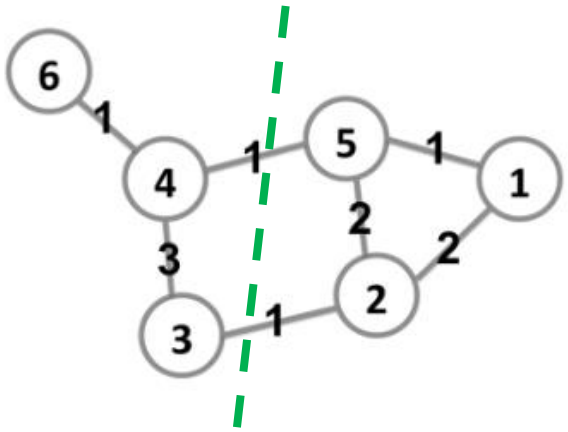
$\min\limits_{\vec{0} \neq \vec{x} \in \mathbb{R}^n} \frac{\vec{x}^T L \vec{x}}{\vec{x}^T \vec{x}}$, subject to $x_i = \pm \frac{1}{\sqrt{n}}$ and $|V_1| = |V_2|$

Note: $\left|\left|\vec{x}\right|\right| = 1$ and $\vec{x}^T \vec{1} = 0$

<u>Approximate Minimization Problem for $V = V_1 \cup V_2$:</u>

$\min\limits_{\vec{0} \neq \vec{x} \in \mathbb{R}^n} \frac{\vec{x}^T L \vec{x}}{\vec{x}^T \vec{x}}$, subject to $\left|\left|\vec{x}\right|\right| = 1,\ \vec{x}^T \vec{1} = 0$

Graph G



Approximate Minimization Problem for $V = V_1 \cup V_2$:

$$\min_{\vec{0} \neq \vec{x} \in \mathbb{R}^n} \frac{\vec{x}^T L \vec{x}}{\vec{x}^T \vec{x}}, \text{ subject to } ||\vec{x}|| = 1,\ \vec{x} \perp \vec{1}$$

Min-Max Theorem

Let L be a symmetric matrix $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$ with corresponding eigenvectors $\vec{x}_1, \vec{x}_2, \ldots, \vec{x}_n$, then

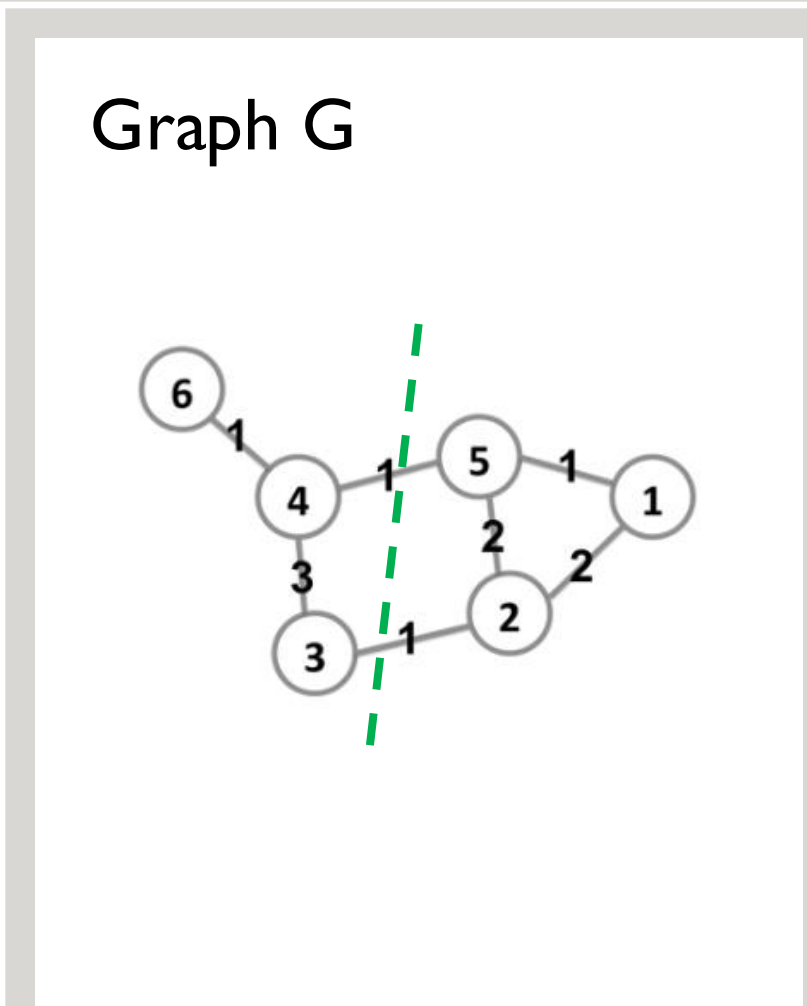
$$\lambda_k = \min_{\vec{x} \perp \vec{x}_1, \ldots, \vec{x}_{k-1}} \frac{\vec{x}^T L \vec{x}}{\vec{x}^T \vec{x}}$$

$$\vec{x}_k = \operatorname*{argmin}_{\vec{x} \perp \vec{x}_1, \ldots, \vec{x}_{k-1}} \frac{\vec{x}^T L \vec{x}}{\vec{x}^T \vec{x}}$$

When k=2:

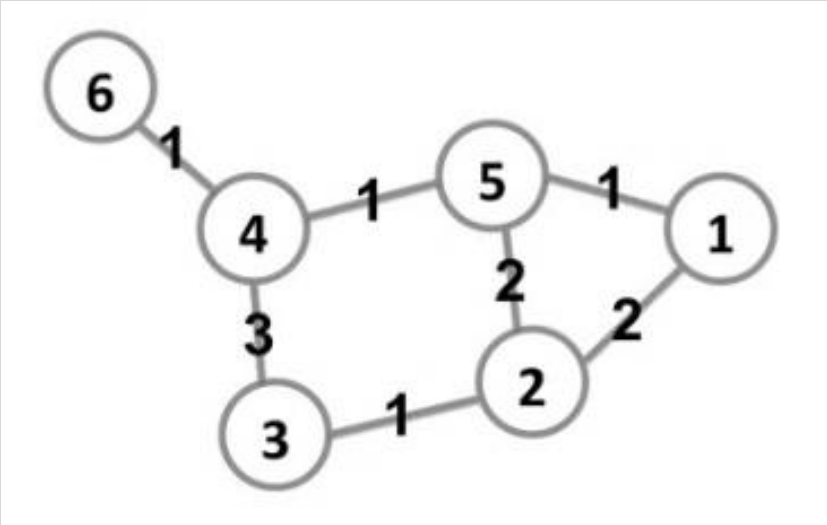
$$\vec{x}_2 = \operatorname*{argmin}_{\vec{x} \perp \vec{x}_1} \frac{\vec{x}^T L \vec{x}}{\vec{x}^T \vec{x}}$$

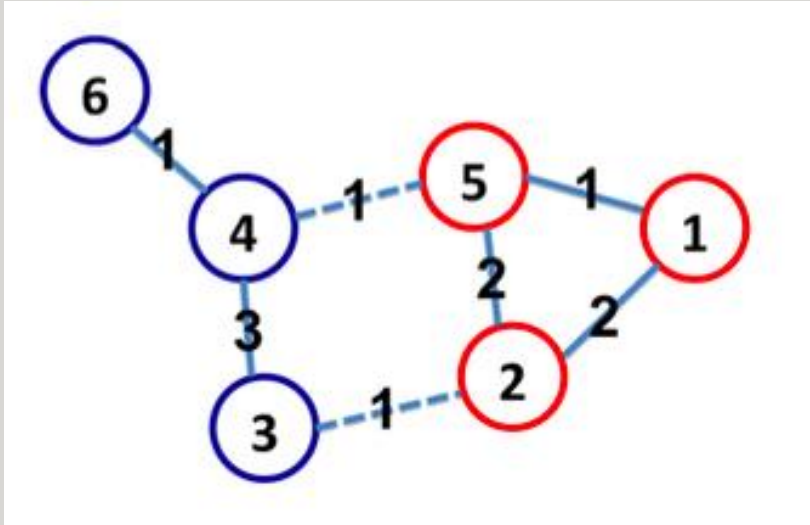
The minimizer is the Fiedler vector $\vec{x}_2$



Compute the Fiedler vector $\vec{x}_2$ corresponding to λ_2 of L

$$L = \begin{bmatrix} 3 & -2 & 0 & 0 & -1 & 0 \\ -2 & 5 & -1 & 0 & -2 & 0 \\ 0 & -1 & 4 & -3 & 0 & 0 \\ 0 & 0 & -3 & 5 & -1 & -1 \\ -1 & -2 & 0 & -1 & 4 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{bmatrix}, \qquad \vec{x}_2 = \begin{bmatrix} -0.41 \\ -0.31 \\ 0.05 \\ 0.16 \\ -0.28 \\ 0.79 \end{bmatrix}$$



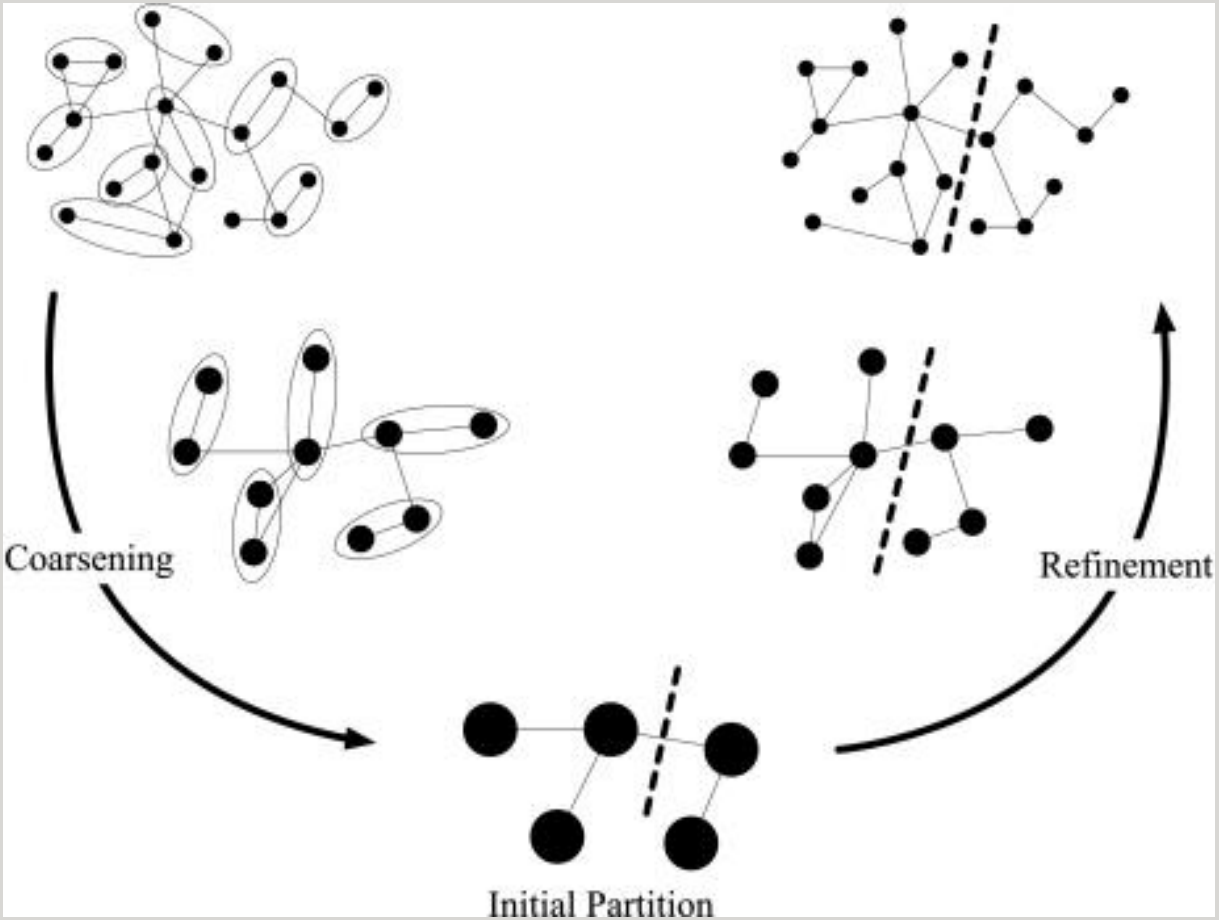


ALGEBRAIC MULTIGRID METHOD (WITH HEAVY EDGE COARSENING)

OUR MOTIVATION

Partitioning a large graph is time-consuming and expensive.

Algebraic Multigrid (AMG) Method: coarsen the graph, solve at a faster speed / lower cost, and refine to our original graph.



COARSENING GRAPH G

Step 1: Group Nodes

$$\{1,2,5\} \rightarrow A$$
$$\{3,4,6\} \rightarrow B$$

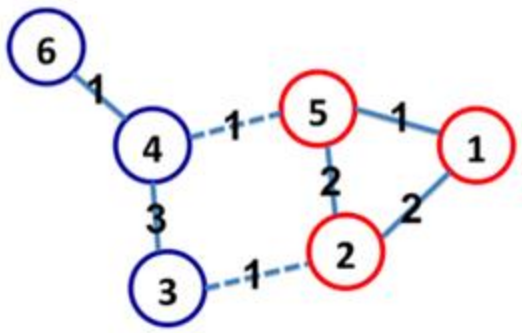
Step 2: Build New Laplacian

$$L(G) = \begin{bmatrix} 3 & -2 & 0 & 0 & -1 & 0 \\ -2 & 5 & -1 & 0 & -2 & 0 \\ 0 & -1 & 4 & -3 & 0 & 0 \\ 0 & 0 & -3 & 5 & -1 & -1 \\ -1 & -2 & 0 & -1 & 4 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{bmatrix}$$

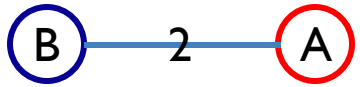
Restriction Matrix $I = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$

$$L(G_c) = I^T L(G) I = \begin{pmatrix} 2 & -2 \\ -2 & 2 \end{pmatrix}$$

Original Graph G



Coarsened Graph G_c



ALGEBRAIC MULTIGRID METHOD (WITH HEAVY EDGE COARSENING)

Algorithm 2.1 Multilevel Cascadic Eigensolver

Input: graph Laplacian matrix $L^0 \in \mathbb{R}^{n_0 \times n_0}$
Output: approximate Fiedler vector $\tilde{y}^{(0)}$

Step 1: Setup Phase
set $i = 0$
while $n_i > 25$ **do**
  $I_i^{i+1} \leftarrow \mathrm{HEC}(L^i)$
  $L^{i+1} = I_i^{i+1} L^i (I_i^{i+1})^T$
  $i = i + 1$
end while
$J \leftarrow i$

Step 2: Coarsest Level Solving Phase
$\tilde{y}^{(J)} \leftarrow \mathrm{PI}(L^J, randn(n_J))$

Step 3: Cascadic Refinement Phase
for $j = J - 1$ **to** 0 **do**
  $\hat{y}^{(j)} = (I_i^{i+1})^T \tilde{y}^{(j+1)}$
  $\tilde{y}^{(j)} \leftarrow \mathrm{PI}(L^j, \hat{y}^{(j)})$
end for

Step 1: Setup Phase

Heavy Edge Coarsening – map each vertex to its heaviest edge

Coarsen graph from G_0 to $G_1, G_2, \ldots, G_J$

Step 2: Coarsest Level Solving Phase

Solve the Fiedler vector for coarse graph G_J by applying power iteration to

$$\mathrm{B} = g * \mathrm{eye(n)} - \frac{g}{n}\overrightarrow{1}\overrightarrow{1}^{\mathrm{T}} - L$$

where $g = ||L||_1$

Step 3: Cascadic Refinement Phase

Extend the Fiedler vector from coarse graph G_J to finer graph G_0

[1] Urschel, John C., et al. "A cascadic multigrid algorithm for computing the Fiedler vector of graph Laplacians." *Journal of Computational Mathematics* (2015): 209-226.

APPLYING AMG TO OUR IMAGE

Input: Digital Image

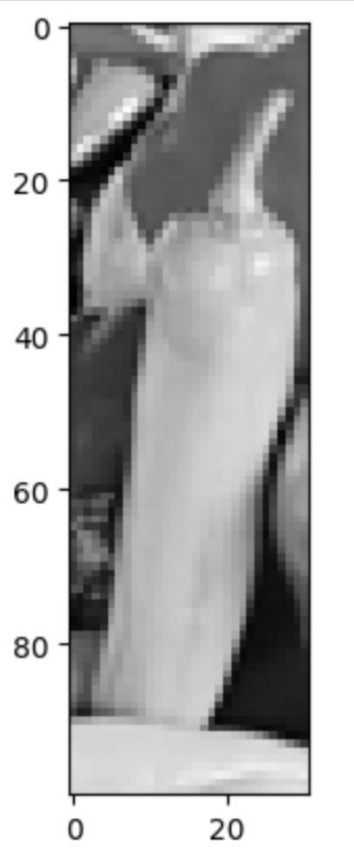
-Compute weights: $w_{ij} = \exp\left(-\frac{\left\|p_i - p_j\right\|^2}{\sigma^2}\right)$ with $\sigma = 5$

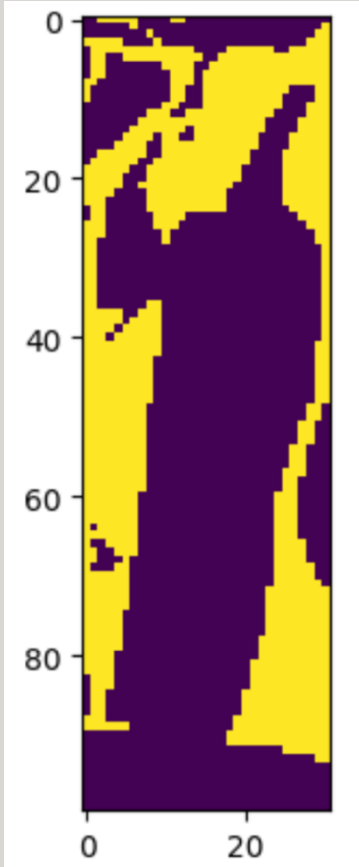
-Apply AMG with HEC to compute Fiedler vector $\vec{x}$

Output: Result Image

If $x_i \geq 0$, set to purple.

If $x_i < 0$, set to yellow.





CONCLUSION

- An image can be represented as a weighted graph
- Graphs can be translated to matrices and solved directly
- Apply AMG method with HEC to find eigenvector efficiently
- Use eigenvector for image segmentation

REFERENCES

Fan RK Chung and Fan Chung Graham.
Spectral graph theory.
Number 92. American Mathematical Soc., 1997.

Miroslav Fiedler.
Algebraic connectivity of graphs.
Czechoslovak mathematical journal, 23(2):298–305, 1973.

Jianbo Shi and Jitendra Malik.
Normalized cuts and image segmentation.
IEEE Transactions on pattern analysis and machine intelligence, 22(8):888–905, 2000.

John C Urschel, Jinchao Xu, Xiaozhe Hu, and Ludmil T Zikatanov.
A cascadic multigrid algorithm for computing the fiedler vector of graph laplacians.
Journal of Computational Mathematics, 33(2):209–226, 2015.

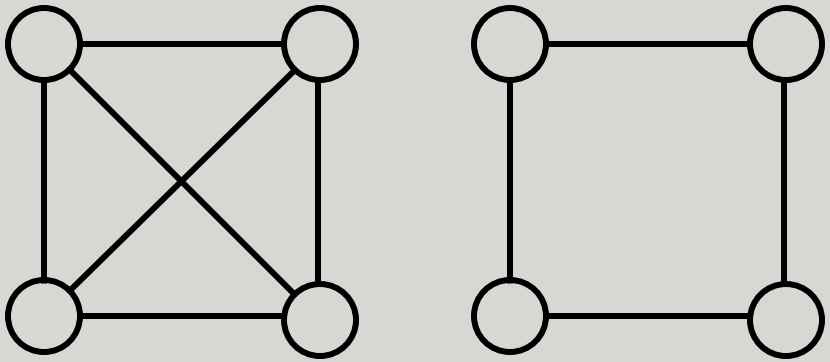
THANK YOU

APPENDIX

Complete Graph (K_n): a simple graph in which there exists an edge between every pair of vertices

Regular Graph: $d(v_i) = r$ for all $v_i \in V$ (r-regular)

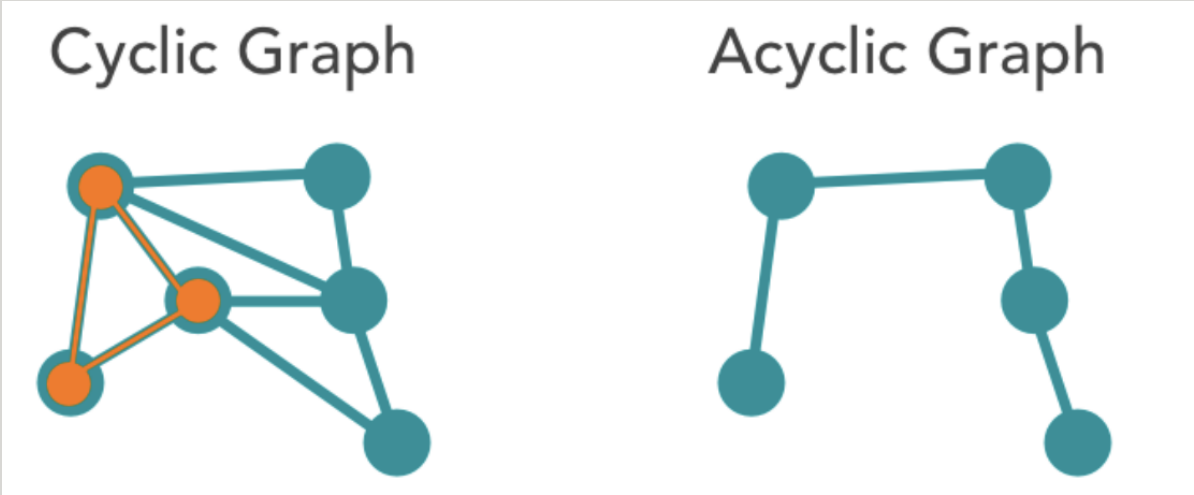
Cycle (C_n): a cycle of n vertices



Empty (Trivial) Graph: a graph with no edges (single vertex)

Consider a sequence of edges $(e_1, e_1, \dots, e_n)$ in a graph G,

- If $e_i = (v_{i-1}, v_i)$ for $i = 1,2, \dots, n$, then the sequence is a **walk**
- If $v_0 = v_n$, a walk is a **closed walk**
- If e_i are distinct, a walk is a **trail**
- If $v_0 = v_n$, a trail is a **closed trail**
- If v_i are distinct, a trail is a **path**
- If $v_0 = v_n$ and $n \geq 3$, a path is a **cycle**
- **Acyclic Graph**: a graph that does not contain a cycle



Connected Vertices: v_i is connected to v_j if there is a path from v_i to v_j

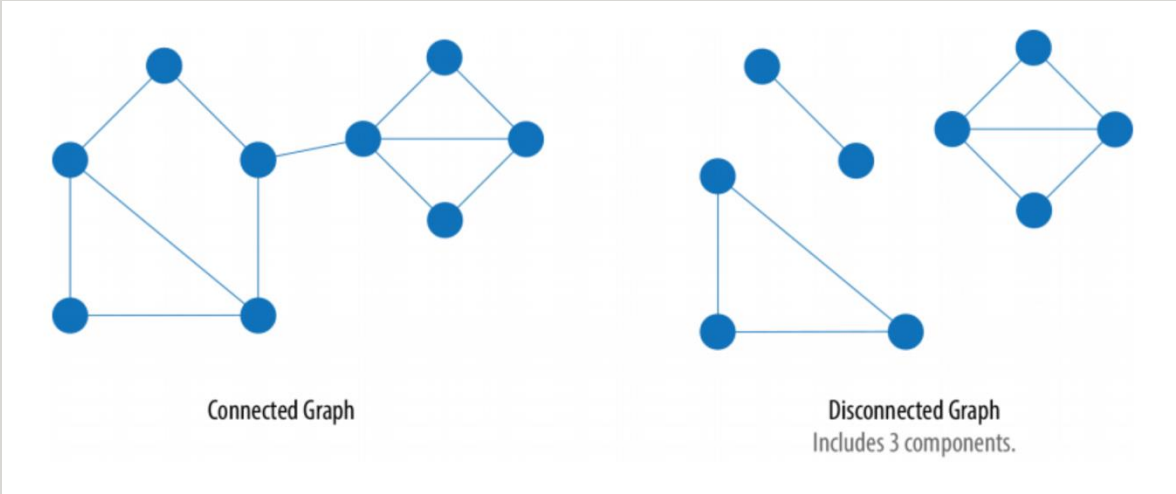
Connected Graph: every two vertices of G are connected

Disconnected Graph: a graph that is not connected

Connected Component: a connected subgraph which is connected to no additional vertices

Lemma: a connected graph with n vertices has at least $n - 1$ edges

Lemma: an acyclic graph with n vertices has at most $n - 1$ edges



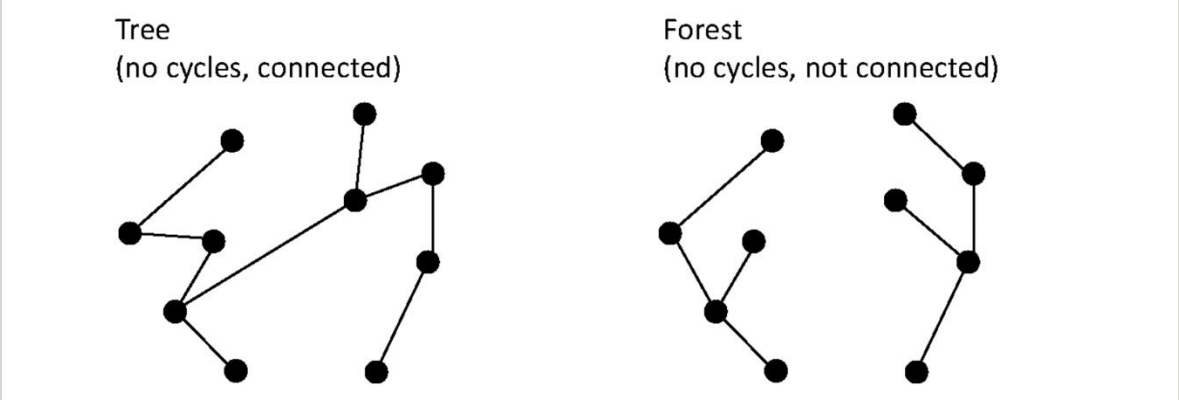
Theorem

Let G be a graph with n vertices, then any two of the following conditions imply the third

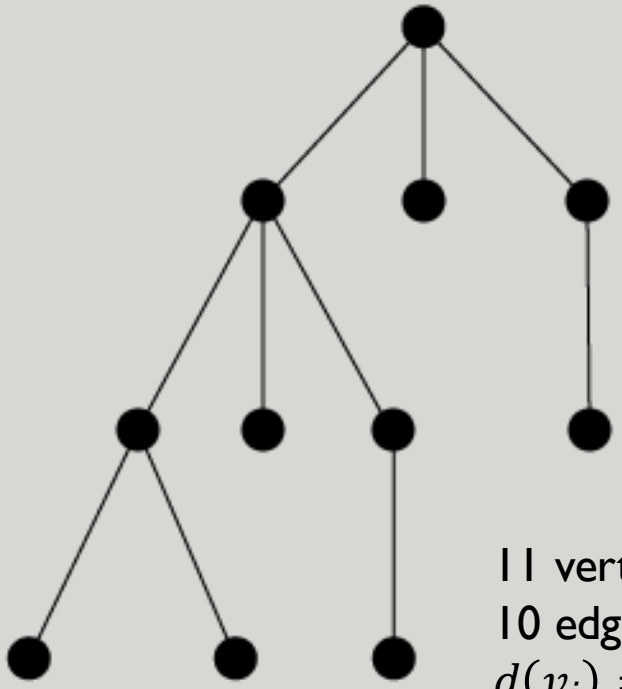
1. G is *connected*
2. G is acyclic
3. G has $n - 1$ *edges*

A graph *T* satisfying all the above conditions is called a **tree**

- **Leaf** v_i: $v_i \in V(T)$ and $d(v_i) = 1$
- **Forest**: a graph whose connected components are trees



Graph *T*



11 vertices
10 edges
$d(v_i) = 1$ for all v_i in *T*

EQUIVALENT MINIMIZATION PROBLEM

Define $\vec{\mathbf{x}} \in \mathbb{R}^n$, where $x_i = \begin{cases} +\dfrac{1}{\sqrt{n}}, & \text{if } i \in \mathcal{V}_1, \\ -\dfrac{1}{\sqrt{n}}, & \text{if } i \in \mathcal{V}_2. \end{cases}$

$$\begin{aligned} \vec{\mathbf{x}}^T L \vec{\mathbf{x}} &= \sum_{i,j \in V} \frac{1}{2} w_{ij} (x_i - x_j)^2 \text{ (next page)} \\ &= \sum_{i \in V_1, j \in V_2} w_{ij} \frac{2}{n} \\ &= \frac{2}{n} \sum_{i \in V_1, j \in V_2} w_{ij} \end{aligned}$$

$$\min \mathbf{cut}(\mathcal{V}_1, \mathcal{V}_2) \iff \min \tfrac{2}{n} \sum_{i \in \mathcal{V}_1, j \in \mathcal{V}_2} w_{ij} = \min \vec{\mathbf{x}}^T L \vec{\mathbf{x}}$$

EQUIVALENT MINIMIZATION PROBLEM

$$
\begin{aligned}
\vec{\mathbf{x}}^T L\vec{\mathbf{x}} &= \sum_{i,j=1}^{n} x_i x_j l_{ij} = \sum_{i=j=1}^{n} l_{ii} x_i^2 + \sum_{i\neq j}^{n} l_{ij} x_i x_j \\
&= \sum_{i=1}^{n} d_i x_i^2 + \sum_{i\neq j}^{n} -w_{ij} x_i x_j \\
&= \sum_{i=1}^{n} (\sum_{j=1}^{n} w_{ij}) x_i^2 - \sum_{i\neq j}^{n} w_{ij} x_i x_j \text{ (by def of } d_i) \\
&= \sum_{i=1}^{n} \sum_{j=1}^{n} w_{ij} x_i^2 - \sum_{i\neq j}^{n} w_{ij} x_i x_j \\
&= \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} w_{ij} x_i^2 + \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} w_{ij} x_i^2 - \sum_{i\neq j}^{n} w_{ij} x_i x_j \\
&= \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} w_{ij} x_i^2 + \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} w_{ij} x_j^2 - \sum_{i\neq j}^{n} w_{ij} x_i x_j \text{ (switch i and j, then use } w_{ij} = w_{ji}) \\
&= \sum_{i,j=1}^{n} \frac{1}{2} w_{ij} (x_i^2 - 2x_i x_j - x_j^2) \\
&= \sum_{i,j=1}^{n} \frac{1}{2} w_{ij} (x_i - x_j)^2
\end{aligned}
$$